Module Specification — pqc_crypto_module v0.1.0

> **Disclaimer**: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

---

1. Module Identification

Field	Value
Module name	`pqc_crypto_module`
Version	0.1.0
Type	Software cryptographic module
Security level	FIPS 140-3 Level 1 (software only)
Language	Rust (2021 edition)
Platform	Linux x86_64/aarch64, macOS aarch64

---

2. Cryptographic Boundary

The boundary is the Rust crate `pqc_crypto_module` located at `crates/pqc_crypto_module/src/`. The Rust module system enforces that external code can only access items marked `pub` in the crate's public API.

Files inside the boundary (11 files)

File	Lines	Role	Contains Crypto
`lib.rs`	~35	Crate root, module re-exports	No
`api.rs`	~82	Single public entry point	Delegates
`mldsa.rs`	~95	ML-DSA-65 sign/verify	Yes
`mlkem.rs`	~130	ML-KEM-768 encaps/decaps	Yes

File	Lines	Role	Contains Crypto
`hashing.rs`	~41	SHA3-256	Yes
`rng.rs`	~61	CSPRNG + continuous test	Yes
`self_tests.rs`	~100	KAT self-tests	Yes (test vectors)
`approved_mode.rs`	~70	FSM state machine	No (control logic)
`types.rs`	~123	Key/sig/hash types with ZeroizeOnDrop	No (data types)
`errors.rs`	~26	CryptoError enum	No (error types)
`legacy.rs`	~120	Non-approved algorithms (gated)	Yes (non-approved)

Boundary enforcement

- Automated test: `tests/crypto_boundary.rs` scans all 189+ source files outside the module and verifies zero raw crypto imports.
- Compile-time: `approved-only` Cargo feature excludes `legacy.rs` entirely via `compile_error!`.

---

3. Public API Surface

All approved operations are accessed exclusively through `pqc_crypto_module::api`.

Control inputs

Function	Input	Effect
`initialize_approved_mode()`	None	Run self-tests, transition FSM to Approved or Error

Data inputs

Function	Input parameters	Description
`sign_message(sk, msg)`	`&MldsaPrivateKey`, `&[u8]`	Message to sign
`verify_signature(pk, msg, sig)`	`&MldsaPublicKey`, `&[u8]`, `&MldsaSignature`	Signature to verify
`sha3_256(data)`	`&[u8]`	Data to hash
`mlkem_encapsulate(pk)`	`&MlKemPublicKey`	Public key for encapsulation

Function	Input parameters	Description
`mlkem_decapsulate(sk, ct)`	`&MlKemPrivateKey, &MlKemCiphertext`	Private key + ciphertext
`random_bytes(n)`	`usize`	Number of random bytes

Data outputs

Function	Output	Description
`generate_mldsa_keypair()`	`MldsaKeyPair { public_key, private_key }`	ML-DSA-65 keypair
`sign_message()`	`MldsaSignature`	3309-byte signature
`verify_signature()`	`()`	Success (or error)
`sha3_256()`	`Hash256`	32-byte digest
`generate_mlkem_keypair()`	`MlKemKeyPair { public_key, private_key }`	ML-KEM-768 keypair
`mlkem_encapsulate()`	`(MlKemCiphertext, MlKemSharedSecret)`	1088-byte ciphertext + 32-byte shared secret
`mlkem_decapsulate()`	`MlKemSharedSecret`	32-byte shared secret
`random_bytes()`	`Vec<u8>`	Random bytes

Status outputs

Condition	Error variant	Meaning
Module not initialized	`CryptoError::ModuleNotInitialized`	FSM in Uninitialized or SelfTesting
Module in error state	`CryptoError::ModuleInErrorState`	FSM in Error (terminal)
Self-test failed	`CryptoError::SelfTestFailed(msg)`	KAT failure during initialization
Invalid key	`CryptoError::InvalidKey(msg)`	Key bytes do not match expected format
Invalid signature	`CryptoError::InvalidSignature`	Signature bytes do not match expected format
Verification failed	`CryptoError::VerificationFailed`	Signature does not verify
Non-approved in FIPS mode	`CryptoError::NonApprovedAlgorithm`	

Condition	Error variant	Meaning
		Legacy function called in Approved state
RNG failure	`CryptoError::RngFailure(msg)`	OS RNG returned an error

4. Approved Algorithms

Algorithm	Standard	Implementation	Key/Output Sizes
ML-DSA-65	FIPS 204	`pqcrypto-mldsa` v0.1.2	PK: 1952 B, SK: 4032 B, Sig: 3309 B
ML-KEM-768	FIPS 203	`pqcrypto-mlkem` v0.1.1	PK: 1184 B, SK: 2400 B, CT: 1088 B, SS: 32 B
SHA3-256	FIPS 202	`sha3` v0.10	Output: 32 B

5. Non-Approved Algorithms

Algorithm	Implementation	Gating
Ed25519	`ed25519-dalek` v2.1	`ensure_not_approved()` runtime guard
SHA-256	`sha2` v0.10	`ensure_not_approved()` runtime guard
HMAC-SHA256	`hmac` v0.12 + `sha2`	`ensure_not_approved()` runtime guard

All non-approved operations return `CryptoError::NonApprovedAlgorithm` when the module is in `Approved` state. Compile-time exclusion via `approved-only` feature.

6. Roles

Role	Description	Authentication
Crypto Officer	Calls `initialize_approved_mode()` at startup	Implicit (first caller)
User	Calls approved crypto services	FSM state check (`require_approved()`)

7. Finite State Machine

```
States: Uninitialized(0), SelfTesting(1), Approved(2), Error(3)

Valid transitions:
  Uninitialized -> SelfTesting   [initialize_approved_mode()
called]
  SelfTesting   -> Approved      [all KATs pass]
  SelfTesting   -> Error         [any KAT fails]

Forbidden transitions:
  Error         -> any           [terminal state]
  Approved      -> Uninitialized [no de-initialization]
  Uninitialized -> Approved      [must pass through SelfTesting]
  Approved      -> SelfTesting   [no re-testing]

Implementation: AtomicU8 with SeqCst ordering.
```

---

8. Self-Tests

KAT	Algorithm	Vectors	Failure action
SHA3-256	SHA3-256	Empty string → known digest; determinism check	Transition to Error
ML-DSA-65	ML-DSA-65	Sign → verify → corrupt → reject; wrong message → reject	Transition to Error
ML-KEM-768	ML-KEM-768	Keygen → encaps → decaps → shared secret match; invalid CT rejection	Transition to Error
RNG	OsRng	Two 32-byte outputs must differ	Transition to Error

---

9. Dependencies (inside boundary)

Crate	Version	Purpose
`pqcrypto-mldsa`	0.1.2	ML-DSA-65 implementation
`pqcrypto-mlkem`	0.1.1	ML-KEM-768 implementation
`pqcrypto-traits`	0.3	Trait interfaces for pqcrypto
`sha3`	0.10	SHA3-256 implementation

Crate	Version	Purpose
`sha2`	0.10	SHA-256 (legacy, non-approved)
`hmac`	0.12	HMAC (legacy, non-approved)
`ed25519-dalek`	2.1	Ed25519 (legacy, non-approved)
`rand`	0.8	OsRng CSPRNG wrapper
`zeroize`	1.7	ZeroizeOnDrop for key material
`thiserror`	1.0	Error type derivation
`hex`	0.4	Hex encoding for digest display

End of module specification.